

Documento de Diseño Arquitectónico

NexaSafe — Componentes, flujo de datos y modelo de datos

NEXASAFE

PROGRAMA

Ingeniería de Sistemas

EQUIPO

Juan Felipe Pineda Cardona · Integrante 2 · Integrante 3

VERSIÓN DEL DOCUMENTO

1.0

INSTITUCIÓN

Fundación Universitaria San Mateo

FECHA DE ENTREGA

22 de septiembre de 2026

DOCUMENTO BASE

PLAN_NEXASAFE.md v1.1

1. Introducción y objetivos de diseño

Este documento describe la arquitectura de software de NexaSafe: sus componentes, el flujo de datos entre ellos, el modelo de datos que sostiene el dominio y las decisiones de diseño que explican por qué el sistema está construido como está. Los objetivos de diseño, en orden de prioridad, son:

1. **Seguridad y privacidad por defecto** — el sistema trata datos sensibles de menores; cada decisión de arquitectura parte de minimizar exposición.
2. **Disponibilidad en condiciones de red pobres** — el trayecto real ocurre en la calle, no en un entorno controlado.
3. **Velocidad de entrega para un equipo de 2–3 personas** — cero infraestructura propia que mantener.
4. **Auditabilidad** — todo incidente debe poder reconstruirse después de ocurrido.

2. Vista de componentes

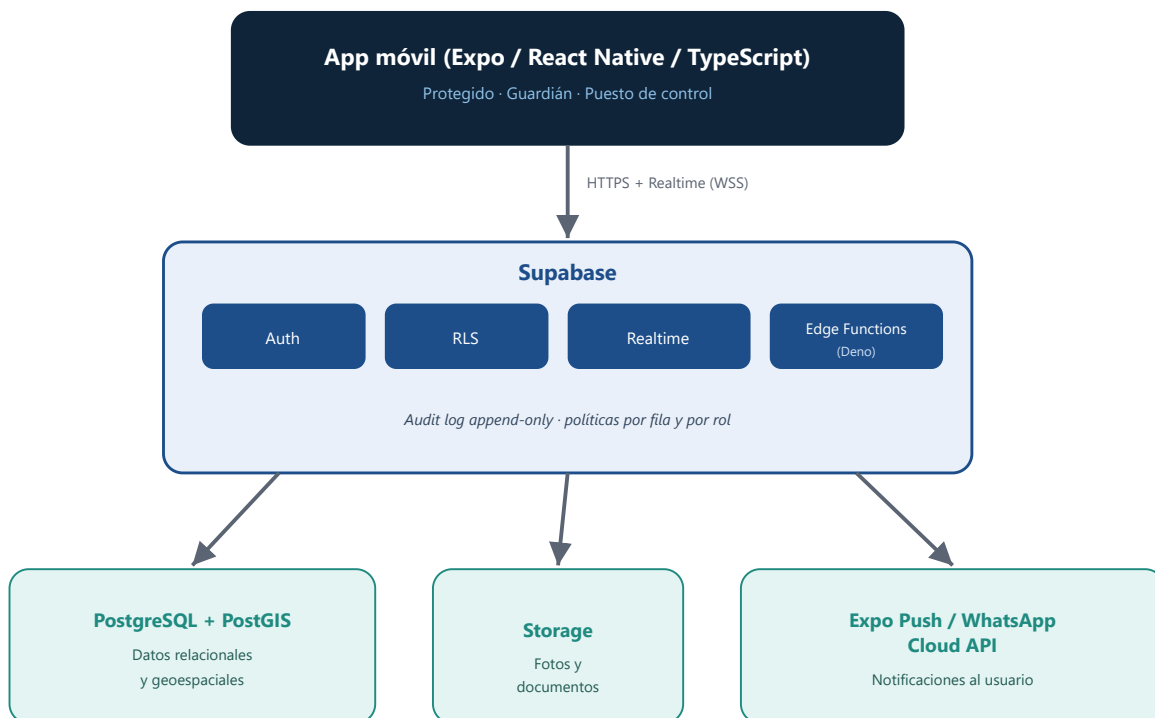


Figura 2.1 — Vista de componentes: app móvil, Supabase como backend gestionado y servicios externos de notificación.

La app móvil es un único binario con tres experiencias condicionadas por el rol autenticado (protegido, guardián o puesto de control), no tres aplicaciones separadas. Supabase concentra autenticación, autorización (RLS), tiempo real y lógica de servidor (Edge Functions) sin que el equipo administre infraestructura propia. PostgreSQL con PostGIS resuelve las consultas geoespaciales del corredor seguro y las geocercas; Storage custodia fotos y documentos; Expo Push y WhatsApp Cloud API son los canales de notificación hacia el dispositivo del usuario final.

3. Stack tecnológico

Capa	Tecnología	Justificación
Móvil	Expo SDK + React Native + TypeScript	Requisito de la asignatura; expo-location, expo-task-manager, expo-secure-store cubren ubicación en segundo plano y almacenamiento cifrado.
Arquitectura móvil	Feature-based: src/features/<f>/{data, domain, presentation} + src/core	Continuidad con la estructura ya definida en el proyecto (ver capítulo 4).
Backend	Supabase — PostgreSQL gestionado + Auth + Realtime + Edge Functions (Deno/TypeScript)	Elimina el montaje, despliegue y mantenimiento de infraestructura propia; autenticación, autorización por RLS y tiempo real vienen resueltos.
Base de datos	PostgreSQL 16 con extensión PostGIS (sobre Supabase)	Consultas geoespaciales para corredor seguro y geocercas.
Tiempo real	Supabase Realtime (WebSocket sobre Postgres)	Difusión de posición a N guardianes sin polling ni servidor propio.
Persistencia local	SQLite (expo-sqlite) + expo-secure-store	Cola offline y custodia del token/PIN.
Notificaciones	Expo Push Notifications + WhatsApp Cloud API (Meta)	Push como canal principal; WhatsApp a contactos de seguridad y puesto de control vía plantilla aprobada.
Infraestructura	Supabase CLI (migraciones versionadas) · GitHub Actions · EAS Build	Reproducible y suficiente para el alcance académico, sin contenedores propios.

4. Arquitectura móvil

La app sigue una organización por características (*feature-based*) con separación en tres capas dentro de cada feature, inspirada en Clean Architecture / MVVM. Esta misma estructura ya está en uso en el prototipo actual del repositorio (feature auth) y se extiende al resto de features del dominio NexaSafe.



Figura 4.1 — Capas dentro de cada feature (presentation → domain → data) y utilidades compartidas en core.

Capa	Responsabilidad
domain	Entidades del negocio y contratos (interfaces de repositorio); no conoce Supabase ni React Native.
data	Acceso a datos: repositorios concretos que implementan los contratos de domain contra Supabase o SQLite local.
presentation	Pantallas y view-models; consume casos de uso de domain, nunca accede a data directamente.
core	Infraestructura transversal: cliente API, almacenamiento seguro, permisos del dispositivo, tema visual y utilidades.

5. Flujo de datos

Se documentan los tres flujos de datos críticos del sistema: trayecto acompañado, alerta de pánico y operación sin conexión. Corresponden a los casos de uso CU-02, CU-04/CU-05 y a la épica E10 del Documento de Requisitos del Sistema.

5.1 Trayecto acompañado con seguimiento en vivo

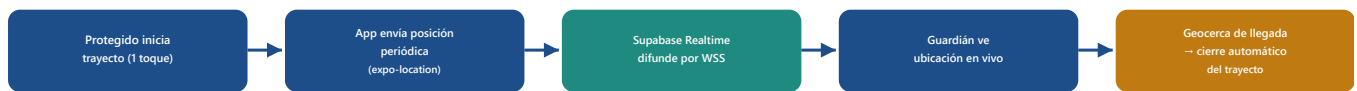


Figura 5.1 — Flujo de datos del trayecto acompañado, de inicio a cierre automático.

5.2 Alerta de pánico



Figura 5.2 — Flujo de datos de la alerta de pánico, con rama de cancelación por PIN.

5.3 Operación degradada sin conexión



Figura 5.3 — Flujo de datos de la resiliencia offline: cola local, reintento y reenvío automático.

6. Modelo de datos

Modelo entidad-relación simplificado del dominio NexaSafe. No incluye tablas puramente técnicas (sesiones, migraciones); se centra en las entidades de negocio necesarias para sustentar los requisitos funcionales del capítulo 3 del Documento de Requisitos del Sistema.

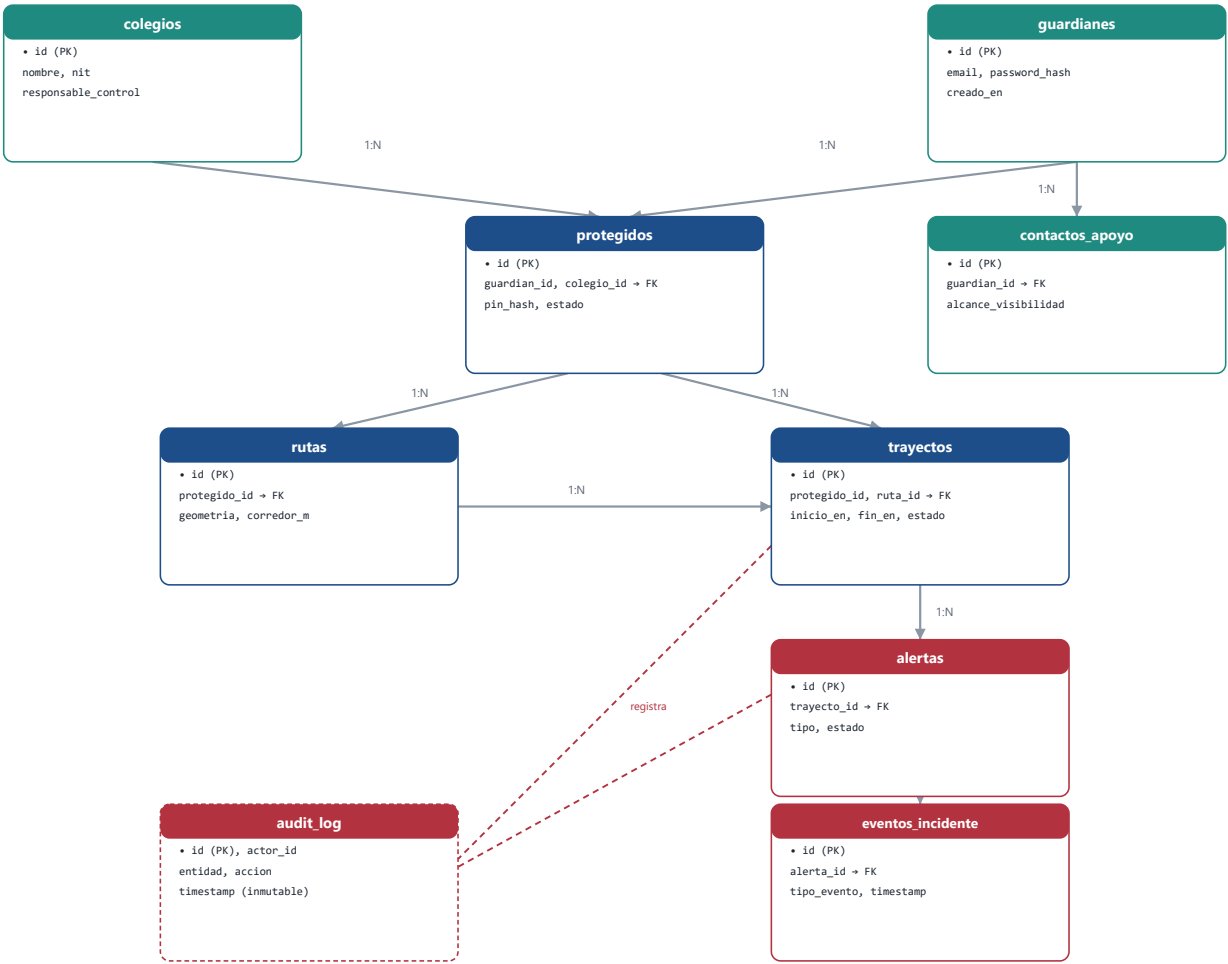


Figura 6.1 — Modelo entidad-relación simplificado de NexaSafe.

6.1 Diccionario de datos resumido

Entidad	Notas de diseño
protegidos.estado	Máquina de estados: PENDIENTE_VALIDACION → ACTIVO → INACTIVO. Solo el colegio puede mover a ACTIVO (RF-03).
trayectos.estado	Máquina de estados: EN_CURSO → DESVIADO → ALERTA o EN_CURSO → CERRADO / VENCIDO.
alertas.tipo	Enumera el origen: MANUAL_BOTON, MANUAL_SACUDIDA, AUTOMATICA_DESVIO.
alertas.estado	Enumera el desenlace: ENVIADA, CANCELADA, ATENDIDA, CERRADA.
eventos_incidente	Tabla append-only; ninguna fila se actualiza ni se borra (RF-32). Sustenta la bitácora auditable.
audit_log	Cross-cutting: registra todo acceso de lectura o escritura a datos de un protegido, con actor, acción y timestamp de servidor.

Retención

trayectos con más de 90 días se purgan automáticamente (RF-39); eventos_incidente se conserva 1 año.

7. Seguridad de la arquitectura

Control	Implementación
Cifrado en tránsito	TLS 1.3 obligatorio + certificate pinning en el cliente móvil.
Cifrado en reposo	Cifrado a nivel de columna para documento, foto y ubicación histórica.
Autenticación	JWT de vida corta (15 min) + refresh token rotativo revocable.
Autorización	RBAC: protegido, guardian, apoyo, institucion — verificada en cada endpoint.
Protección del PIN	Hash Argon2id, nunca en texto plano, almacenado en expo-secure-store.
Anti-abuso	Rate limiting por IP y por usuario en endpoints de alerta y autenticación.
Anti-tampering	Detección de root/emulador en modo release.
Minimización	Solo se persiste ubicación con trayecto o alerta activos; fuera de eso no se recolecta.
Trazabilidad	Todo acceso a datos de un menor queda en audit_log con actor, acción y timestamp.

8. Decisiones arquitectónicas clave

Decisión	Razonamiento
Backend en Supabase en lugar de un backend propio	Un equipo de 2–3 personas no puede sostener un backend a medida (FastAPI + Redis + Docker) con la misma calidad de seguridad que una plataforma gestionada con Auth, RLS y Realtime ya endurecidos.
Vista institucional dentro de la misma app, sin dashboard web	Reduce la superficie de código a mantener y de ataque a asegurar; el puesto de control necesita mapa y lista de alertas, no un panel administrativo completo.
WhatsApp Cloud API como canal secundario, no principal	El número de prueba solo soporta 5 destinatarios verificados (riesgo R5); el push permanece como canal principal sin ese límite.
Disparo de pánico por sacudida en vez de botón físico externo	Evita depender de hardware adicional que el usuario deba comprar y llevar; se limita a modo trayecto con la app en primer plano para reducir falsos positivos.
Escalamiento a la autoridad simulado, no integrado	La integración real con la línea 123 requiere convenio interinstitucional fuera del alcance académico; el sistema genera y registra el reporte sin transmitirlo a un organismo real.

9. Estructura del repositorio

```
nexasafe/
├── .github/
│   ├── workflows/          # ci.yml, nightly-security.yml, release.yml
│   ├── ISSUE_TEMPLATE/     # user-story.md, bug.md, security-finding.md
│   └── pull_request_template.md
├── apps/
│   └── mobile/              # Expo + React Native + TypeScript
│       └── src/
│           ├── core/        # api, storage, permissions, theme, utils
│           └── features/
│               ├── auth/     {data, domain, presentation}
│               ├── profile/
│               ├── routes/
│               ├── tracking/
│               ├── alerts/
│               ├── network/
│               ├── institution/ # vista del puesto de control
│               └── incidents/
├── supabase/
│   ├── migrations/         # esquema versionado
│   ├── functions/          # Edge Functions (Deno): notify-push, notify-whatsapp, escalate
│   └── policies/            # políticas RLS documentadas y probadas
├── docs/
│   ├── architecture/
│   ├── security/           # threat-model.md, risk-register.md, exceptions.md
│   ├── privacy/            # pia.md, politica-tratamiento.md
│   └── scrum/               # sprint-XX/{planning, review, retro}.md
├── .pre-commit-config.yaml
├── .gitleaks.toml
├── semgrep.yml
└── README.md
```